

Differences Between Software Engineering and Computer Science

According to the concepts discussed in the textbooks by Ian Sommerville and Roger S. Pressman, Computer Science and Software Engineering are closely related but have different focuses.

Basis	Computer Science	Software Engineering
Definition	Study of computation, algorithms, and the theoretical foundations of computing.	Application of engineering principles to the development, operation, and maintenance of software.
Primary Focus	Theory, algorithms, data structures, and computation.	Design, development, testing, deployment, and maintenance of software systems.
Objective	To understand how computers solve problems.	To build reliable, efficient, and maintainable software.
Nature	More theoretical and scientific.	More practical and engineering-oriented.
Concerned With	Algorithms, programming languages, artificial intelligence, operating systems, theory of computation.	Software processes, requirements engineering, software design, testing, project management, quality assurance.

Differences Between Software Engineering and Computer Science

Approach	Focuses on discovering and analyzing computing principles.	Focuses on applying principles to develop software products.
Output	New computing theories, algorithms, and models.	Software products and software solutions.
Scale of Work	Can involve small programs and theoretical models.	Usually deals with large and complex software systems.
Project Management	Generally not a major concern.	An essential component.
Quality Assurance	Less emphasis.	Strong emphasis on software quality, reliability, and maintenance.

Differences Between Software Engineering and Computer Science

Computer Science

Computer Science answers:

- How does a computer work?
- How can problems be solved efficiently?
- Which algorithm is best?
- How can artificial intelligence be developed?

Example:

- Developing a new sorting algorithm or studying machine learning theories.

Differences Between Software Engineering and Computer Science

Software Engineering

Software Engineering answers:

- How do we build a software system?
- How do we manage a software project?
- How do we ensure software quality?
- How do we maintain software after deployment?

Example:

- Developing a Banking System, Hospital Management System, Facebook-like application, or E-commerce platform.

Differences Between Software Engineering and Computer Science

Relationship Between Computer Science and Software Engineering

Computer Science



Provides theories,
algorithms and principles

Software Engineering



Applies those principles
to build software systems

Differences Between Software Engineering and Computer Science

Key Point from Somerville

- Computer Science focuses on theory and fundamentals of computation, whereas Software Engineering focuses on practical aspects of developing and delivering useful software systems.

Differences Between Software Engineering and System Engineering

Aspect	Software Engineering	System Engineering
Definition	The discipline of designing, developing, testing, and maintaining software systems.	The interdisciplinary approach to design, integrate, and manage complex systems (hardware + software + people + processes).
Focus	Mainly focuses on software products .	Focuses on the entire system including hardware, software, and environment.
Entity Scope	Narrow scope (software only).	Broad scope (whole system lifecycle).
Goal	To produce high-quality, efficient, and reliable software.	To ensure the whole system meets user needs and functions as a unified whole.
Components	Programs, applications, algorithms, data structures.	Hardware, software, networks, humans, procedures.
Nature of Work	Implementation and maintenance of software.	Requirement analysis, system design, integration, validation, and management.

Differences Between Software Engineering and System Engineering

Aspect	Software Engineering	System Engineering
Complexity Handling	Handles software complexity.	Handles system-level complexity (integration of multiple subsystems).
Lifecycle Coverage	Software Development Life Cycle (SDLC).	System Development Life Cycle (full system lifecycle).
Example	Developing a banking application, mobile app, or website.	Designing an ATM system, aircraft system, or smart city system.
Team Involvement	Software developers, testers, designers.	Engineers from multiple domains (software, electrical, mechanical, etc.).
Output	Software product.	Fully functional integrated system.

Differences Between Software Engineering and System Engineering

Simple Understanding

Software Engineering → “Builds the software part only.”

System Engineering → “Builds and manages the whole system including software + hardware + environment.”

Key Difference in One Line

Software Engineering = “Software only solution”

System Engineering = “Complete system solution (software + hardware + people + process)”

Professional Software Development

- Many people can write computer programs, but writing a program is different from developing professional software.
- A student may write a program that works on their computer, but professional software must:
- Work correctly for many users.
 - Be reliable and secure.
 - Be easy to maintain.
 - Meet customer requirements.
 - Be delivered on time and within budget.
 - Continue operating for many years.

Therefore, Professional Software Development is much more than coding.

Professional Software Development

- Professional Software Development is the **systematic, disciplined, and organized process of developing, delivering, operating, and maintaining** high-quality software systems that **satisfy customer and business requirements** while following engineering principles, professional ethics, and industry standards.
- According to Ian Sommerville,
professional software development is **concerned not only with producing software** but also with **managing** the entire software engineering process, ensuring software **quality**, and **supporting** the software throughout its operational life.

Software Engineering Diversity

- According to Ian Sommerville,
software engineering is diverse because there is no universal process, method, or technique that is appropriate for all types of software systems. The choice of development process, tools, and practices depends on the nature of the software and its requirements.
- Key Idea:
“ One software engineering approach cannot be applied effectively to every software project.”

Software Engineering Diversity

Why Does Diversity Exist?

Different software systems have different requirements because they vary in:

- Size and complexity
- Purpose and application domain
- Number of users
- Performance requirements
- Security needs
- Reliability requirements
- Operating environment
- Hardware platform
- Budget and schedule
- Regulatory and legal constraints

As a result, developers must choose methods that best suit the project rather than relying on a single universal approach.

Internet Software Engineering

- According to Ian Sommerville,

Internet software engineering emphasizes **rapid development, continuous evolution, service-oriented design, scalability, security, and frequent deployment**, since Internet applications must **adapt quickly** to changing user needs and technologies.

- Internet Software Engineering is the application of software engineering principles, methods, and tools to the development, deployment, operation, and maintenance of software systems that are delivered and accessed through the Internet.
- These applications typically use web technologies, cloud services, databases, and network communication to provide services to users across different devices and locations.

Internet Software Engineering

Characteristics of Internet Software Engineering

- A. Web-Based Access
- B. Continuous Availability
- C. Rapid Development and Deployment
- D. Large Number of Users
- E. Distributed Architecture
- F. Security Requirements
- G. Frequent Requirement Changes